# Переносимое ТЗ: совместная работа с текстом документа (просмотр / комментирование / предложения / редактирование) + ролевая модель

> **Статус: ПЕРЕНОСИМЕ ТЗ — не опис цього проєкту.** Документ написано, щоб той самий модуль можна було відтворити **в іншому проєкті** з нуля; він навмисно самодостатній і дублює частину `SPEC-document-editor.md`.
>
> Для роботи над **цим** репозиторієм читай `docs/ARCHITECTURE.md`, а не цей файл. Тримається в репозиторії як зліпок для перевикористання; з реалізацією синхронізувати не потрібно.

> Самодостаточное техническое задание для агента-разработчика **другого проекта**. Описывает, как реализовать «окно текста документа»: просмотр, комментирование, предложения правок, прямое редактирование, импорт Word (.docx), создание нового документа с нуля — **и ролевую модель (RBAC)**, которая всё это гейтит.
>
> Документ говорит «что и как». Имена файлов в скобках — ориентир из эталонной реализации (Next.js 14 App Router + tRPC + Prisma/PostgreSQL + TipTap); стек можно заменить аналогами, сохранив поведение и контракты.

---

## 0. Что строим (в двух абзацах)

У некоторой сущности (в эталоне — «стандарт» или «документ», у вас — любая сущность, у которой есть редактируемое **тело**) хранится форматированный текст (HTML). С этим текстом совместно работают несколько ролей:

- одни **редактируют напрямую** (WYSIWYG, автосохранение),
- другие **только предлагают правки** (suggestions) и **комментируют** (inline + ветки),
- третьи **только просматривают**.

Текст можно **набрать с нуля**, **импортировать из Word (.docx)** или **загрузить как файл-документ**. Кто что может — определяет **ролевая матрица** (раздел 1). Всё это — в одном UI с тремя режимами: **Просмотр / Предложения / Редактирование** (раздел 5).

**Эталонные файлы** (для справки): `src/components/StandardBodyEditor.tsx` (главный компонент, ~1600 строк), `src/components/ui/RichTextEditor.tsx` (обёртка TipTap), `src/components/InlineComments.tsx` (overlay выделения + список),

`src/server/routers/{suggestion,document,inlineComment,comment}.ts`, `src/lib/rbac.ts`, `src/lib/standardBody.ts` (split/join блоков), `src/lib/sanitizeHtml.ts`, `src/app/api/standards/[id]/import-body/route.ts` (mammoth .docx→HTML).

---

## 1. Ролевая модель (RBAC) — реализовать целиком

Модуль бессмысленен без прав, поэтому ролевую модель закладывайте **сразу**.

### 1.1 Уровни ролей

- **Глобальные роли** (на пользователе): `ADMIN`, `DIRECTOR` (руководство/надзор), `USER`.
  - `ADMIN` — short-circuit allow на всё.
  - `DIRECTOR` — видит **всё** (read везде) + действия из своей колонки матрицы, применяются ко всем областям, **без** привязки к членству.
- **Скоуп-роли** (членство пользователя в группе/контейнере; в эталоне — «рабочая группа»): `LEADER`, `DEPUTY`, `SECRETARY`, `MEMBER`, `GUEST`. У одного пользователя может быть членство в нескольких контейнерах с разными ролями.

> Если у вас нет «групп», скоуп = сам документ/проект/организация. Главное — сохранить модель «глобальная роль ∪ роль-в-скоупе → набор действий».

### 1.2 Действия (actions) — то, что проверяется

Минимум для этого модуля:

| Действие | Что разрешает | Дефолтные роли |
|---|---|---|
| `doc:editBody` | Прямое редактирование тела **без согласования** + принять/отклонить предложение | LEADER, DEPUTY, SECRETARY |
| `doc:upload` | Создать документ, загрузить/импортировать файл, редактировать метаданные | LEADER, DEPUTY, SECRETARY |
| `doc:setCurrent` | Пометить версию документа «актуальной» | LEADER, DEPUTY |
| `doc:delete` | Удалить документ/файл | LEADER, DEPUTY, SECRETARY |
| `comment:add` | Создавать предложения и inline-комментарии, ставить реакции | все участники (incl. MEMBER) |
| `doc:editMeta` | Редактировать карточку сущности (метаданные), **не тело** | LEADER, DEPUTY, SECRETARY |

(в эталоне эти действия названы `standard:editBody`, `document:upload`, `document:setCurrent`, `document:delete`, `comment:add`, `standard:editMeta` — переименуйте под свой домен.)

### 1.3 Как устроена проверка прав

- **Хардкод-дефолты** в коде: `PERMISSIONS: Record<action, role[]>` (`src/lib/rbac.ts`).
- **БД-оверрайды**: таблица `RolePermission { role, action, allowed }`. Оверрайд **побеждает** дефолт. Пустого оверрайда нет — отсутствие записи = дефолт. Кэш в памяти процесса, `ensureLoaded()` прогревается один раз в middleware.
- **Единая функция** `can(user, action, scopeId): boolean` — **одна и та же на клиенте и сервере** (чистый модуль без импорта БД; на сервере в неё через `registerOverrideLookup` прокидывается доступ к кэшу оверрайдов, на клиенте — null, т.е. клиент использует дефолты, а сервер — авторитетный источник).

```
function can(user, action, scopeId) {
  if (user.globalRole === 'ADMIN') return true;
  if (user.globalRole === 'DIRECTOR') {
    if (READ_ACTIONS.includes(action)) return true;
    if (effectiveAllowed('DIRECTOR', action)) return true; // колонка директора
  }
  const role = roleInScope(user, scopeId); // ADMIN→LEADER-level; иначе из memberships
  if (!role) return false;
  return effectiveAllowed(role, action); // override ?? default
}
```

- **`seesAll(user)`** — отдельный хелпер для «видит все контейнеры»: `true` для ADMIN/DIRECTOR и (в эталоне, по требованию заказчика) для любого `SECRETARY`. Используется в `list`-запросах: `where = seesAll ? {} : { scopeId: { in: myScopeIds } }`.
- **Админка матрицы**: таблица «роль × действие» с тогглами, пишет в `RolePermission`. Удаление оверрайда (возврат к дефолту) — удалять строку (держать таблицу разреженной).

### 1.4 Сессия и паритет клиент/сервер (важно для UX)

- Сессия (JWT) несёт `globalRole` + `memberships[] {scopeId, role}`. Клиент строит из этого `userCtx` и вызывает тот же `can()`, чтобы **скрывать недоступные кнопки** (но не как защиту).
- **Сервер — единственный источник истины**: каждая мутация/`byId`/`list` проверяет право и кидает `FORBIDDEN`/пустой результат при отказе (см. раздел 9 и 10).
- **Без перелогина**: смена роли/права применяется на следующем запросе — `session` callback дочитывает `globalRole` + `memberships` (и `isActive`) из БД на каждый рендер; не ревалидировать по фокусу окна (это даёт гонки и «слетающую» сессию).
- **Деактивация**: `authorize` (логин) и сессия должны проверять `user.isActive` — иначе выключенный пользователь продолжает работать.

---

## 2. Модель данных (Prisma-ориентир)

### 2.1 Тело сущности

```
Entity {                        // напр. Standard / Document-owner / Article
  bodyText        String?       // HTML тела (для «документа» поле может зваться bodyHtml)
  bodyUpdatedAt   DateTime?
  bodyUpdatedById String?
  ...
}
```

- `bodyText` / `bodyHtml` — **HTML**, который эмитит WYSIWYG. Лимит на запись ~200 КБ.
- Полиморфизм: предложения и inline-комментарии могут относиться к сущности **или** к вложенному документу. На уровне приложения — XOR: заполнено ровно одно из `standardId` / `documentId`.

### 2.2 Document (загруженный/созданный файл-документ)

```
Document {
  id            String   @id @default(cuid())
  standardId    String                          // владелец (scope берётся через него)
  type          enum (DRAFT_STANDARD|TECH_SPEC|FEEDBACK|MEETING_MINUTES|AGENDA|ATTACHMENT|FINAL)
  filename      String
  s3Key         String?                         // null для созданных «пустых» (только bodyHtml)
  sizeBytes     Int      @default(0)
  version       String
  isCurrent     Boolean  @default(false)         // только одна «актуальная» на сущность
  allowEdits    Boolean  @default(false)         // разрешено ли коллаборативное редактирование т
  bodyHtml      String?                         // редактируемое тело документа (если allowEdits
  bodyUpdatedAt DateTime?
  bodyUpdatedById String?
  uploadedById  String
  createdAt     DateTime @default(now())
  standard      Standard @relation(..., onDelete: Cascade)
}
```

### 2.3 Suggestion (предложение правки)

```
Suggestion {
  id            String  @id @default(cuid())
  standardId    String?                          // XOR
  documentId    String?                          // XOR
  authorId      String
  paragraphIndex Int                              // индекс блока в теле
  originalText  String                            // HTML-снапшот блока на момент создания (re-an
  proposedText  String                            // предлагаемый HTML (для DELETE — '')
  operation     enum (REPLACE | INSERT_AFTER | DELETE)
  status        enum (PENDING | ACCEPTED | REJECTED) @default(PENDING)
  rationale     String?
  resolvedAt    DateTime?
  resolvedById  String?
  resolveNote   String?
  createdAt     DateTime @default(now())
  reactions     SuggestionReaction[]              // { userId, type: LIKE|DISLIKE } — совещательны
  @@index([standardId, status])
}
```

### 2.4 InlineComment (комментарий к выделению) + ветка ответов

```
InlineComment {
  id            String  @id @default(cuid())
  standardId    String?                          // XOR
  documentId    String?                          // XOR
  authorId      String
  paragraphIndex Int
  startOffset   Int
  endOffset     Int
  selectionText String                            // снапшот выделенного текста (re-anchor)
  body          String
  status        enum (OPEN | RESOLVED) @default(OPEN)
  resolvedById  String?
  resolvedAt    DateTime?
  createdAt     DateTime @default(now())
  replies       InlineCommentReply[] (onDelete: Cascade)
  @@index([standardId, status])
}
InlineCommentReply { id, commentId, authorId, body, createdAt }
```

### 2.5 (опц.) Comment — обычные треды на уровне сущности

Плоский список с `parentId` (2 уровня вложенности), строится в дерево на клиенте. Удалять — автор или LEADER/ADMIN. Это **отдельная** от inline сущность (комментарии «ко всему документу», а не к

фрагменту).

### 2.6 Аудит

`ActivityLog { userId, action(CREATE|UPDATE|DELETE|STATUS_CHANGE), entity, entityId, before:Json?, after:Json?, note?, createdAt }`. Каждая мутация тела/предложения/документа пишет запись. Покрытие — проверять в CI (статический анализ: каждая мутация содержит `logActivity(...)` или явно в exempt-списке).

> ⚠️ Если деплой-пайплайн делает `prisma db push --accept-data-loss` на каждом старте — меняйте схему только аддитивно (nullable-поля). Все дочерние связи к сущности — `onDelete: Cascade`, чтобы удаление чистило suggestions/comments/documents; **файлы в объектном хранилище (S3) cascade не чистит** — удаляйте их явно best-effort в `delete`-процедуре.

---

## 3. Жизненный цикл документа (создание / ввод / импорт / загрузка)

Это то, что должно «просто работать» из UI.

### 3.1 Создать новый документ и ввести текст с нуля

- Кнопка «Почати з нуля» / «Новий документ» (для `doc:upload`).
- Создаёт запись `Document` (или просто инициализирует `bodyHtml=''`, `s3Key=null`, `allowEdits=true`) → открывает **режим Редактирование** (WYSIWYG) на пустом теле.
- Пользователь печатает → автосохранение (раздел 6.2). Файла в S3 нет — это «нативный» документ.

(эталон: `document.createEmpty` с правом `document:upload`.)

### 3.2 Импорт .docx в тело (заменить текст текущего тела)

- Кнопка «Імпортувати з Word» в режиме редактирования.
- Клиентская валидация: только `.docx` (иначе toast-ошибка).
- `POST /api/<entity>/[id]/import-body` (multipart, право `doc:editBody`): сервер конвертирует .docx → HTML через **mammoth**, **сохраняет стили** (см. 4.2), возвращает `{ html, warnings? }`.
- Если тело уже непустое → **подтверждение** (ConfirmModal) с предупреждением: будут удалены «осиротевшие» предложения/inline-комментарии (их якоря указывают на старый текст). По подтверждению — `replaceBody` (атомарно: заменить тело + удалить suggestions/inlineComments этой сущности в одной транзакции).
- После импорта **перемонтировать WYSIWYG** на новый HTML (бамп `seedKey`, см. 6.2), чтобы то, что видит пользователь, совпало с сохранённым.

### 3.3 Загрузить .docx/.pdf как файл-документ (вложение, не тело)

Два пути; выберите один как основной:

- **Прокси-аплоад (рекомендуется, проще и безопаснее):** `POST /api/<entity>/[id]/documents` (multipart). Сервер сам кладёт файл в S3, **сам ставит Content-Type** и создаёт `Document`. Право `doc:upload`.
- **Presigned PUT (если нужен прямой аплоад в S3):** `document.getUploadUrl` (право `doc:upload`) → клиент PUT-ит файл по подписанному URL → `document.confirmUpload` / `registerMetadata` (право `doc:upload`!) создаёт `Document`-запись. ⚠️ На presigned-скачивании всегда ставить `Content-Disposition: attachment`, иначе .pdf с HTML-нагрузкой может выполниться в браузере.

Документы образуют список с версиями; одна помечена `isCurrent` (право `doc:setCurrent`, делать транзакцией: снять флаг со старой + поставить новой). Если у документа `allowEdits=true`, его `bodyHtml` редактируется тем же модулем, что и тело сущности (полиморфизм по `documentId`).

### 3.4 Экспорт

- Тело → **.docx** (lib `docx`) и **PDF** (`@react-pdf/renderer`) с сохранением форматирования (шрифты, размеры в pt, выравнивание, таблицы, ссылки). Для кириллицы в PDF — встроить статический шрифт (напр. NotoSans). Единый базовый шрифт — Times New Roman.

### 3.5 Пустые состояния

- Для `doc:editBody`: предложить «Імпортувати .docx» **или** «Почати з нуля».
- Для остальных: «Текст ще не додано» (inline, не громоздкий placeholder).

---

## 4. Формат тела и разбиение на блоки

Тело — HTML от WYSIWYG. **Блок** = top-level элемент (`<p>`, `<h1..3>`, `<ul>/<ol>`, `<blockquote>`, `<pre>`, `<table>`). Предложения и inline-комментарии адресуются **индексом блока**.

### 4.1 `splitBlocks(html): string[]` / `joinBlocks(blocks): string`

- **Клиент (есть DOM):** `DOMParser` → `Array.from(root.children).map(el => el.outerHTML)`.
- **Сервер/SSR (нет DOM):** ручной сканер по тегам с подсчётом глубины вложенности.
- 🔴 **КРИТИЧНО — void-элементы.** При подсчёте глубины **НЕ увеличивать** глубину для HTML void-тегов: `area base br col embed hr img input link meta param source track wbr`. Иначе `<col>` внутри таблиц (WYSIWYG эмитит `<table><colgroup><col><col>…`) «не закрывается», глубина не возвращается к 0 → **весь текст после первой таблицы слипается в один блок** → индексы съезжают → принятие предложений падает с ложным «оригинальный текст не найден».
- **Клиентский и серверный сплиттер ОБЯЗАНЫ давать одинаковые границы блоков.** Это инвариант, на котором держится re-anchoring (раздел 8).
- Нормализация: legacy plain-text (без тегов) прозрачно мигрируется в `<p>…</p>`.

### 4.2 Сохранение стилей при импорте .docx

- Сохранять: размер шрифта (в **pt**), выравнивание, таблицы, ссылки, базовые начертания.

- Размер шрифта переносить инлайново `style="font-size: Npt"` (маппинг style-имён mammoth → span с классом → пост-обработка в pt).

---

## 5. Три режима и выбор режима по правам

Переключатель — сегмент-контрол в шапке. Режим — в локальном стейте; **дефолт вычисляется лениво от прав**, чтобы не «прыгать», пока грузится сессия:

```
defaultMode = canEditBody ? 'edit' : canComment ? 'suggest' : 'view'
mode = pickedMode ?? defaultMode        // pickedMode=null до ручного переключения
```

Доступность сегментов: **Просмотр** — всем; **Предложения** — `comment:add`; **Редактирование** — только `doc:editBody`. Если доступен только «Просмотр» — переключатель не показывать.

| Режим | Что показывает | Взаимодействие |
|---|---|---|
| Просмотр | Read-only статья по блокам | Видны подсветки существующих комментариев; создавать/принимать нельзя |
| Предложения | Read-only статья + правая панель | Кнопки предложений на блок; inline-комментарии через выделение; принять/отклонить (для editBody); реакции |
| Редактирование | Всегда-включённый WYSIWYG + панель | Прямые правки + автосейв; импорт .docx; правая панель = список «Зміни/Коментарі» (без создания через выделение) |

---

## 6. Режим «Редактирование» (WYSIWYG)

### 6.1 Редактор

- Всегда-включённый WYSIWYG (TipTap/ProseMirror). Расширения: bold/italic/underline/strike, H1–H3, ul/ol, blockquote, code, **font-family + font-size (pt, 10–24)**, цвет текста, выравнивание (left/center/right/justify), ссылки, **таблицы**.
- Санитизация контента — строгой схемой редактора (принимаются только объявленные узлы/марки → инъекция скрипта через UI невозможна по построению). **НО см. раздел 10** — серверная санитизация обязательна, т.к. API можно дёрнуть в обход редактора.
- Базовый шрифт тела — Times New Roman; инлайновые стили перекрывают.

### 6.2 Автосохранение (без кнопки «Сохранить»)

- Debounce ~**1200 мс** после последнего ввода → мутация `updateBody`.
- **Flush при размонтировании** (смена режима / уход со страницы) — несохранённое не теряется.
- Сравнивать с последним сохранённым (через `ref`) — не слать пустые сейвы.
- Статус в шапке: «Збереження…» / «Збережено автоматично · дата · кто».

- **Контракт монтирования:** редактор сидируется контентом **один раз** на `seedKey`. Рутинный поллинг (тело меняют другие) **не должен** перемонтировать редактор и затирать набор. `seedKey` бампится **только** при полной замене тела (импорт .docx). При бампе — сбросить baseline и отменить отложенный автосейв.

### 6.3 Закреплённая (sticky) панель форматирования

- Тулбар форматирования — `sticky` к верху, не уезжает при скролле длинного документа.
- Над ним — sticky-шапка действий (раздел 7). Тулбар прибивается **сразу под** шапкой: `toolbarTop = высота шапки действий`. Две полосы **стекаются**, без перекрытия и щели.

---

## 7. Шапка действий (sticky) и стекинг

- Слева: переключатель режимов + статус (edit → автосейв; read-only → «Оновлено …, N відкритих правок»).
- Справа: «Експортувати .docx»; в режиме редактирования ещё «Імпортувати з Word».
- Шапка `sticky` (на странице `top:0`; в модалке — под её заголовком). `z-index` шапки выше тулбара; тулбар прибит сразу под шапкой.

---

## 8. Режим «Предложения»: поток правок и комментариев

### 8.1 Статья по блокам

- Тело рендерится **поблочно** (read-only), каждый блок в обёртке `data-paragraph-idx={idx}` (для скролла и привязки выделения).
- На блоке при наведении — мини-панель (для `comment:add`): **Замінити** (REPLACE, сидируя исходным HTML блока) / **Додати після** (INSERT_AFTER) / **Видалити** (DELETE).
- Внизу — «Запропонувати новий параграф у кінці» (INSERT_AFTER к последнему блоку).

### 8.2 Inline-комментарии (через выделение)

- Overlay поверх статьи слушает выделение текста (mouseup) внутри `data-paragraph-idx` → плавающий pill «💬 Коментувати» → композер → создать комментарий, привязанный к `paragraphIndex + startOffset/endOffset + selectionText`.
- Создание возможно **только на read-only статье** (в WYSIWYG своя модель выделения; overlay туда не вешать — это вешает рендер). Поэтому комментарии «живут» в режиме «Предложения».
- В «Просмотре» overlay смонтирован, но создание выключено (`canComment=false`) — видны только существующие подсветки.
- **Ре-анкеринг:** если текст сместился — искать якорь по `selectionText`; не найден → пометить «потерял якорь», не падать.

### 8.3 Правая панель «Зміни / Коментарі»

- Две вкладки со счётчиками (открытых/всего). «Зміни» — список предложений (pending + последние решённые), клик → скролл к блоку с подсветкой. «Коментарі» — inline-комментарии + ответы + «Вирішено». Панель `sticky` со своей прокруткой. Тот же компонент — в режиме редактирования (но без overlay создания).

### 8.4 Реакции

- На предложение — 👍/👎 (совещательные, без кворума), счётчики видны всем (`comment:add`).

---

## 9. Принятие / отклонение предложений

### 9.1 Модалка-предпросмотр (diff)

- «Прийняти» → модалка с дифом: REPLACE — пословный diff (red strike = удалено, green = добавлено)
  - серый оригинал; INSERT_AFTER — новый блок (зелёный); DELETE — удаляемый блок (красный, зачёркнутый); + автор и обоснование. Кнопки «Прийняти зміну» / «Скасувати». Diff — общий компонент (карточка и модалка используют один).

### 9.2 Серверная `accept` — устойчивость к конкуренции (re-anchor)

Заново читать текущее тело и ре-анкерить **по содержимому**, не слепо по индексу:

1. Взять блок по `paragraphIndex`; сравнить plain-text (strip тегов + нормализация пробелов/энтити) с `originalText`. Совпало → применить здесь.
2. Иначе — найти блок, чей plain-text == `originalText`. Найден → применить по нему.
3. Не найден:
   - DELETE → блок уже удалён → пометить решённым (no-op);
   - INSERT_AFTER → добавить `proposedText` в конец (намерение не теряем);
   - REPLACE → реальный конфликт → понятная ошибка «оригінальний текст параграфа №N вже не знайдено (хтось його змінив), перегляньте правку та створіть нову».

- Детект «уже применено»: если `proposedText` уже есть в каком-то блоке — пометить решённым, не дублировать.
- Применение → `joinBlocks` (выкинуть пустые) → сохранить тело (`bodyUpdatedAt/By`) → статус предложения ACCEPTED → аудит → уведомить автора.

### 9.3 Модель конкурентности

Last-write-wins для тела + re-anchor для предложений. Кворум/блокировки не нужны. (Опц.: версия тела `bodyVersion` с optimistic-lock и 409 при гонке — если нужна строгость.)

---

## 10. Серверный API + безопасность (учтите уроки!)

### 10.1 Процедуры (tRPC-ориентир) и права

| Процедура | Право | Назначение |
|---|---|---|
| `suggestion.list({standardId\|documentId})` | membership/seesAll | список предложений (поллинг ~5с) |
| `suggestion.create(target, paragraphIndex, originalText, proposedText, operation, rationale?)` | `comment:add` | создать предложение |
| `suggestion.react({suggestionId, type\|null})` | `comment:add` | 👍/👎 / снять |
| `suggestion.accept({id, note?})` | `doc:editBody` | принять (см. 9.2) |
| `suggestion.reject({id, note?})` | `doc:editBody` | отклонить |
| `suggestion.updateBody(target, bodyText)` | `doc:editBody` | прямое сохранение тела (автосейв) |
| `suggestion.replaceBody(target, bodyText)` | `doc:editBody` | атомарная замена тела + чистка осиротевших |
| `inlineComment.list/create/reply/setResolved/delete` | `comment:add` (delete: автор/editBody) | inline-комментарии |
| `document.createEmpty / getUploadUrl / confirmUpload / registerMetadata` | `doc:upload` | создать/загрузить документ |
| `document.list({standardId})` | membership/seesAll | список документов |
| `document.setAsCurrent / updateMeta` | `doc:setCurrent` / `doc:upload` | версии/метаданные |
| `document.delete` | `doc:delete` | удалить (+ best-effort S3 cleanup) |
| `POST /api/<entity>/[id]/import-body` | `doc:editBody` | mammoth .docx→HTML |
| `POST /api/<entity>/[id]/documents` | `doc:upload` | прокси-аплоад файла в S3 |

### 10.2 Security — обязательные правила (грабли из прод-аудита)

1. 🔴 **Серверная санитизация HTML.** Тело и тексты предложений приходят через API и потом рендерятся через `dangerouslySetInnerHTML`. Схема WYSIWYG чистит **только на клиенте** —

прямой вызов API её обходит. **Санитизируйте на сервере перед записью** ( `updateBody` , `replaceBody` , `suggestion.create.{originalText,proposedText}` , любое `bodyHtml` ). Эталон: `isomorphic-dompurify` , профиль `html` (без SVG/MathML), запрет `<img>/<iframe>/<script>` и не-http(s)/mailto/relative URL — это нейтрализует `<img src=x onerror=...>` . ⚠️ jsdom (внутри isomorphic-dompurify) не бандлится сервером Next — пометьте пакет `serverComponentsExternalPackages` .

2. 🔴 **RBAC на КАЖДОЙ процедуре.** Любой `byId` / `list` /мутация обязаны проверять право или членство ( `can(...)` либо `seesAll || isMember` ), иначе IDOR: чужой документ/предложение читается по cuid. `list` -фильтры по `scopeId` из инпута **пересекать** с доступными скоупами (не доверять клиентскому списку id).
3. **Лимиты входа:** `bodyText ≤ 200KB` , `proposedText/originalText ≤ 20KB` , `cuid()` -валидация id.
4. **Атомарность:** `replaceBody` , `setAsCurrent` , многошаговые операции — в `$transaction` .
5. **Аудит** на каждую мутацию (раздел 2.6).
6. Каждая мутация — отдельная проверка права на сервере, 403 при отказе (UI лишь прячет кнопки).

---

## 11. Уведомления и прочее

- **Уведомления (колокольчик):** «вам предложили правку», «ваша правка принята/отклонена», упоминания в комментариях. Тексты обезличены корректно (без подстановки чужого имени).
- **Подтверждения и сообщения в UI:** только in-app компоненты — `ConfirmModal` (с опциональным type-to-confirm для опасных действий) и `toast` (success/error/info). **Никаких нативных `confirm()` / `alert()` .**
- Удаление документов/тел — обратимо через аудит/архив (физического удаления тела нет; «удаление документа» — отдельная привилегия `doc:delete` + чистка S3).

---

## 12. Edge cases / нефункциональные

- Не перемонтировать WYSIWYG на рутинном поллинге (затрёт ввод) — только на импорте ( `seedKey` ).
- Не ломать поток предложений/комментариев для пользователей **без** `doc:editBody` .
- Большие документы (сотни блоков, таблицы): сплиттер/рендер держат нагрузку; не монтировать per-параграф WYSIWYG-overlay (это «вешает» рендер → отсюда раздельные режимы).
- Тема: токены light/dark, без hardcoded `bg-white` / `slate-*` (не переключаются в dark).

---

## 13. Критерии приёмки

1. Пользователь с `doc:editBody` открывает тело → сразу режим «Редагування», WYSIWYG, автосейв работает, статус обновляется, кнопки «Зберегти» нет.

2. Тулбар форматирования и шапка действий **закреплены** и не уезжают/не перекрываются при скролле.
3. Переключение режимов доступно по правам; дефолт не «прыгает» на загрузке; при только-просмотре переключатель скрыт.
4. **Новый документ с нуля:** «Почати з нуля» → пустой WYSIWYG → ввод текста → автосейв.
5. **Импорт .docx в тело:** заменяет тело, чистит осиротевшие предложения/комментарии, перемонтирует редактор; сохранены шрифты/размеры(pt)/таблицы/выравнивание.
6. **Загрузка файла-документа** (.docx/.pdf) создаёт версию; «актуальная» — одна; скачивание — attachment.
7. В «Предложениях»: создаются предложения (заменить/добавить/удалить) и inline-комментарии через выделение; правая панель со счётчиками; реакции.
8. Принятие предложения открывает **модалку-предпросмотр с дифом**; по подтверждению применяется; работает **на документе с таблицами** (нет ложного «параграф не знайдено» — сплиттер клиент=сервер, void-`<col>` не ломает индексы).
9. Пользователь **без** `doc:editBody` видит Просмотр/Предложения, прямого редактирования и принятия нет; пользователь без `comment:add` — только Просмотр.
10. **Security:** прямой API-вызов `updateBody` с `<img src=x onerror=...>` сохраняет санитизированный HTML (скрипт не исполняется); чужой `suggestion/inlineComment/document` по cuid не читается без членства; каждая мутация пишет аудит.
11. Ролевая матрица конфигурируется в админке (тоггл «роль × действие»), оверрайд побеждает дефолт, применяется без перелогина; деактивированный пользователь не может ни войти, ни действовать.

---

*Это переносимое ТЗ, выжимка из реально работающего модуля (StandardBodyEditor + suggestion/document/inlineComment-роутеры + RBAC) эталонного проекта. Пиксельные offset'ы и тайминги — ориентировочные, подбираются под рендер. Имена сущностей/действий адаптируйте под свой домен, сохранив контракты и инварианты (особенно: одинаковый сплиттер клиент↔сервер, серверную санитизацию и RBAC на каждой процедуре).*